TechSage AI

Software Design Document (SDD)

Architecture, Component Design & Data Flow

Document Version 1.0

GenLink Hacks 2026 | April 2026
Submitted to: GenLink Nonprofit, Austin TX

Companion document to the TechSage AI SRS v1.0

1. Introduction

1.1 Purpose

This Software Design Document (SDD) describes the architecture, component structure, data flows, and technology decisions for TechSage AI. It translates the requirements defined in the SRS into a concrete blueprint for implementation. It is intended for the development team, reviewers, and GenLink staff evaluating the project for senior center deployment.

1.2 Design Goals

- Simplicity first: every design decision prioritizes ease of use over feature richness
- Zero-install: the entire application runs in a browser with no downloads or accounts
- AI-augmented: Claude API powers natural language answers, scam analysis, and personalized guidance
- Resilient: core features (tutorials, quiz) work offline; AI features degrade gracefully
- Privacy-preserving: no personal data stored beyond the browser session

1.3 Scope

This document covers: system architecture, technology stack, component design, API integration, data models, user interface design principles, and the 6-day development sprint plan.

2. System Architecture

2.1 Architecture Overview

TechSage AI uses a lightweight client-server architecture with a thin serverless backend. The frontend is a React single-page application (SPA). The only server-side component is a single serverless function that proxies requests to the Claude API, keeping the API key secure.

Layer	Technology	Responsibility
Frontend (Client)	React + Vite, Tailwind CSS	UI rendering, voice I/O, state management, offline quiz/tutorials
Voice Layer	Web Speech API (browser-native)	ASR (speech-to-text) + TTS (text-to-speech), no external service
API Proxy (Server)	Vercel Serverless Function (Node.js)	Receives frontend requests, appends API key, forwards to Claude
AI Backend	Anthropic Claude API (claude-sonnet-4)	Language understanding, answer generation, scam analysis
Deployment	Vercel (free tier)	Static hosting + serverless functions, custom domain via .vercel.app
Static Data	JSON files in /src/data/	Tutorial steps, quiz scenarios — loaded client-side, no API needed

2.2 System Architecture Diagram

```
[ Senior User's Browser ]
Voice Input / Button Click
↓
[ React SPA — Frontend ]  Voice Layer (Web Speech API)
HTTPS POST /api/chat
↓
[ Vercel Serverless Function — /api/chat.js ]
Appends API Key + System Prompt → forwards to Anthropic
↓
[ Anthropic Claude API  (claude-sonnet-4) ]
```

2.3 Data Flow — Voice Assistant Query

Step	Actor	Action
1	User	Taps microphone button on TechSage homepage
2	Web Speech API	Captures spoken audio, streams transcription to React state
3	React frontend	Displays transcribed text; user confirms or edits
4	React frontend	Sends POST /api/chat with { message, language, mode: 'assistant' }
5	Serverless proxy	Validates request, injects system prompt + API key, calls Claude API
6	Claude API	Returns plain-language response JSON
7	Serverless proxy	Returns { reply } to frontend
8	React frontend	Displays reply in large text; Web Speech API reads it aloud
9	User	Hears and reads the answer; can ask a follow-up

2.4 Data Flow — Scam Detection

Step	Actor	Action
1	User	Pastes or speaks a suspicious message on the Scam Detector tab
2	React frontend	Sends POST /api/chat with { message, mode: 'scam' }
3	Serverless proxy	Injects scam-detection system prompt; calls Claude API
4	Claude API	Returns structured verdict: { verdict, redFlags[], recommendation }
5	React frontend	Renders green (Safe) or red (Scam/Suspicious) verdict card + red flags
6	Web Speech API	Reads verdict aloud

3. Technology Stack

3.1 Frontend

Technology	Version	Reason for Choice
React	18+	Component-based UI; easy to manage state for voice/chat interactions
Vite	5+	Fast builds, instant HMR, simple config for student teams
Tailwind CSS	3+	Utility-first styling; easy to enforce large fonts and high contrast
Web Speech API	Browser native	No extra service or cost; works in Chrome and Edge
React Router	6+	Tab navigation between Home, Scam Detector, Tutorials, Quiz

3.2 Backend (Serverless)

Technology	Version	Reason for Choice
Node.js	18 LTS	Standard Vercel runtime for serverless functions
Vercel Functions	Latest	Free tier, zero config, same repo as frontend
Anthropic SDK	0.20+	Official JS SDK; handles retries, streaming, type safety

3.3 Deployment & Tooling

Technology	Purpose
Vercel	Hosting (free tier) + serverless functions + HTTPS + CDN
GitHub	Version control + Vercel auto-deploy on push
ESLint + Prettier	Code quality; consistent formatting for team

4. Component Design

4.1 Frontend Component Tree

```
App.jsx
├── Layout.jsx          (header with logo, text-size controls, language toggle)
├── HomePage.jsx        (large welcome, quick-question buttons, mic button)
│   ├── MicButton.jsx   (start/stop voice recording, animated state)
│   ├── ChatBox.jsx     (displays conversation history)
│   └── QuickQuestions.jsx  (pre-set question chips)
├── ScamDetector.jsx    (paste/speak a message → verdict card)
│   ├── MessageInput.jsx
│   └── VerdictCard.jsx (green/red, red flags list, recommendation)
├── ScamQuiz.jsx        (scenario quiz, score tracker)
│   └── QuizCard.jsx    (scenario text, Safe/Scam buttons, feedback)
├── Tutorials.jsx       (grid of lesson cards)
│   └── TutorialViewer.jsx  (step-by-step with next/back, read-aloud)
```

```
└── Settings.jsx        (font size, language — accessible from all pages)
```

4.2 Component Specifications

4.2.1 MicButton

Property	Detail
Size	Minimum 80x80px; circular; high-contrast colors
States	Idle (blue), Recording (pulsing red), Processing (spinning), Error (orange)
Behavior	On tap: starts SpeechRecognition. On tap again or silence: stops and submits
Accessibility	aria-label changes per state; keyboard-operable via Enter/Space

4.2.2 VerdictCard

Property	Detail
Safe state	Green background, large checkmark icon, 'This appears safe' in bold 20pt
Scam state	Red background, warning icon, verdict label, list of red flags, recommendation
Suspicious state	Amber background, caution icon, partial red flags listed
TTS	Automatically reads verdict + first red flag aloud on render

4.2.3 TutorialViewer

Property	Detail
Navigation	Large Previous / Next buttons (min 56px height), step counter (Step 2 of 5)
Content	Step title (22pt bold), step description (18pt), optional illustration
Voice	'Read this step aloud' button using TTS
Progress	Visual step dots at top showing completion

4.3 Custom Hooks

Hook	Purpose
useSpeechRecognition()	Wraps Web Speech API ASR; returns { transcript, isListening, start, stop, error }
useSpeechSynthesis()	Wraps Web Speech API TTS; returns { speak, stop, isSpeaking }
useClaudeAPI()	Handles fetch to /api/chat; returns { sendMessage, response, loading, error }
useSettings()	Manages fontSize and language in sessionStorage; returns { fontSize, language, setFontSize, setLanguage }

5. API Design

5.1 Serverless Endpoint: POST /api/chat

Request schema

```
{
  "message": "string — the user's question or message",
  "mode": "assistant" | "scam",
  "language": "en" | "ta"
}
```

Response schema — mode: assistant

```
{
  "reply": "string — plain-language answer from Claude"
}
```

Response schema — mode: scam

```
{
  "verdict": "SAFE" | "SUSPICIOUS" | "SCAM",
  "redFlags": ["string", "string"],
  "recommendation": "string"
}
```

Error response

```
{
  "error": "string — friendly error message for display to user"
}
```

5.2 Rate Limiting

- Maximum 20 requests per session (tracked in sessionStorage)
- If limit reached: display message 'You've used all your questions for today. Try again tomorrow or ask a family member for help.'
- Serverless function validates presence and length of message field (max 1000 chars)

> **Security note:** The API key is stored only as a Vercel Environment Variable. It is never sent to the client or included in the JavaScript bundle.

6. Data Models

6.1 Tutorial Data Schema (/src/data/tutorials.json)

```
[
  {
    "id": "email",
    "title": "How to send an email",
    "icon": "📧",
    "difficulty": "beginner",
    "steps": [
      {
        "stepNumber": 1,
        "title": "Open your email app",
```

```
        "description": "Find the Mail or Gmail icon on your phone. It looks like an envelope.",
        "imageAlt": "Mail app icon on home screen"
      }
    ]
  }
]
```

6.2 Quiz Scenario Schema (/src/data/quizScenarios.json)

```
[
  {
    "id": "q001",
    "scenario": "You receive a text: URGENT — your bank account has been suspended. Call 1-800-555-1234 now.",
    "correctAnswer": "SCAM",
    "explanation": "Banks never suspend accounts by text message or ask you to call an unknown number.",
    "redFlags": ["Urgency language (URGENT)", "Unknown phone number", "Threatening consequence"]
  }
]
```

7. User Interface Design Principles

7.1 Senior-Friendly Design Rules

Design Rule	Specification	Rationale
Font size	18pt body minimum, 22pt headings, adjustable +/- via toolbar	Age-related vision decline; WCAG 1.4.4
Touch targets	Minimum 56x56px for all interactive elements	Reduced fine motor control in seniors
Color contrast	4.5:1 minimum ratio; never rely on color alone	Color vision changes with age; WCAG 1.4.3
Button labels	Text labels on all buttons, never icon-only	Icons are not universally recognized
Feedback	Every action produces visible + audible confirmation	Reduces anxiety about 'did it work?'
Error messages	Plain language; always include a suggested next step	Technical errors confuse and discourage seniors
Navigation depth	Maximum 2 taps to any feature from home screen	Reduces cognitive load and chance of getting lost
Language	Tamil and English; defaults to browser language setting	Local language increases comfort and trust

7.2 Color System

Element	Color	Purpose
Primary action buttons	#1F4E79 (dark blue)	High contrast, trustworthy, calm
Mic button — idle	#2E75B6 (medium blue)	Inviting, clearly interactive
Mic button — recording	#C00000 (red)	Clear 'recording now' signal
Scam verdict — safe	#375623 text on #E2EFDA bg	Green = safe, universally understood
Scam verdict — scam	#843C0C text on #FCE4D6 bg	Red/orange = danger, high contrast
Background	#FFFFFF with #F8FBFF secondary	Maximum contrast for text
Body text	#1A1A1A (near black)	4.5:1+ contrast on white background

8. Security Design

8.1 API Key Protection

- API key stored in Vercel Environment Variables, never in source code or client bundle
- Serverless proxy is the only component that touches the key
- CORS restricted to the production domain only

8.2 Input Sanitization

- All user input stripped of HTML/script tags before sending to Claude API
- Maximum input length: 1000 characters (enforced client and server side)
- Claude API system prompt instructs the model to stay on topic and refuse harmful requests

8.3 Privacy

- No cookies, analytics, or tracking scripts
- Session state stored in sessionStorage only (cleared when browser tab closes)
- Messages sent to Claude API are subject to Anthropic's data handling policy (no training use per API terms)

> **Privacy statement for users:** This app does not save any of your questions or messages. Everything is forgotten when you close the tab.

9. Six-Day Development Sprint

Day	Focus	Deliverables
Day 1 — Apr 13	Setup & Structure	GitHub repo, Vercel deploy, React + Tailwind scaffold, routing between 4 tabs, basic large-button layout live on URL
Day 2 — Apr	Voice Layer	MicButton component, Web Speech API ASR working, TTS

Day	Focus	Deliverables
14		reading back text, text fallback for Safari
Day 3 — Apr 15	Claude API Integration	Serverless proxy at /api/chat, useClaudeAPI hook, end-to-end voice question → Claude answer → spoken reply
Day 4 — Apr 16	Scam Features	Scam Detector tab (paste/speak + verdict card), Scam Quiz with 10 scenarios, score tracker
Day 5 — Apr 17	Tutorials + Polish	6 tutorial cards with step viewer, Tamil language toggle, A+/A- font controls, accessibility audit
Day 6 — Apr 18	Demo & Submit	Record 2–3 min demo video, take screenshots, write README, final Vercel deploy, Devpost submission

Hackathon deadline: April 18, 2026 @ 9:30 AM IST. Day 6 work must begin by 6:00 AM to allow buffer for final submission.

10. Testing Strategy

Test Type	Coverage	Method
Unit tests	useClaudeAPI hook, input sanitization, quiz scoring logic	Vitest (built into Vite)
Component tests	MicButton states, VerdictCard rendering for all 3 verdicts	React Testing Library
Manual testing	Full user flow on real devices: Android phone, iPad, Windows laptop	Manual checklist
Accessibility check	Color contrast, touch target size, keyboard navigation	Chrome DevTools Lighthouse audit
Senior user testing	At least one walkthrough with a non-technical adult to validate UX	Informal observation

11. Deployment Architecture

11.1 Vercel Project Structure

```
techsage-ai/
├── public/                    Static assets (icons, tutorial images)
├── src/
│   ├── components/            React UI components
│   ├── hooks/                 Custom hooks (voice, API, settings)
│   ├── data/                  tutorials.json, quizScenarios.json
│   ├── pages/                 HomePage, ScamDetector, Tutorials, Quiz
│   └── App.jsx + main.jsx
├── api/
│   └── chat.js                Serverless proxy function
├── .env.local                 ANTHROPIC_API_KEY (never committed)
├── vercel.json                Routing config
```

```
└── README.md
```

11.2 Environment Variables

Variable	Where Set	Value
ANTHROPIC_API_KEY	Vercel Dashboard → Environment Variables	sk-ant-... (secret)
VITE_APP_NAME	vercel.json or .env	TechSage AI
VITE_MAX_REQUESTS	vercel.json or .env	20